



National Autonomous University of Mexico



Faculty of Engineering

Syntax Analyzer (Parser)

Compilers
Group 05

Team 03

320068234
424039512
424153520
321264257
320336658

April 30, 2026
Semester 2026-2

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Motivation	3
1.3	Objectives	3
2	Theoretical Framework	4
2.1	Compiler Structure	4
2.2	Context-Free Grammar	4
2.3	Context-Free Grammars in Compilers	6
2.4	Associativity	6
2.5	Precedence	6
2.5.1	BNF	6
2.5.2	Types of Parsers	7
2.6	Semantic Analysis	8
3	Product Overview: ParseFlow	9
3.1	Features	9
3.2	Token Normalization	9
3.3	File Handling	9
4	Development	10
4.1	Architectural Overview	10
4.2	Core Component Modeling	10
4.2.1	Atomic Units: Productions and Items	10
4.2.2	The DFA State Machine	10
4.3	Grammar Pre-processing	11
4.3.1	Grammar Loading and Identification	11
4.3.2	FIRST Set Calculation	11
4.4	DFA Construction Logic	11
4.4.1	The Closure Operation	11
4.4.2	The Goto Operation	11

4.5	Collection Orchestration and Optimization	11
4.5.1	Canonical LR(1) Collection	11
4.5.2	LALR(1) Merging (The Core Logic)	12
4.6	Parsing Table Synthesis	12
4.7	The Shift-Reduce Parsing Engine	12
4.7.1	Parse Tree Visualization	12
4.8	Grammar Implementation Details	13
4.8.1	Grammar Structural Analysis	15
4.9	Semantic Rules Specification	16
4.9.1	Syntax-Directed Translation Validation	16
4.9.2	Semantic Analysis Component	16
4.10	User Interface: ParseFlow Application	16
4.11	Main Execution Flow	17
5	Results	18
5.1	Case 1: Parsing Success & SDT Verified	19
5.1.1	Example 1	19
5.1.2	Example 2	21
5.2	Case 2: Parsing Success & SDT Error	22
5.2.1	Example 1	22
5.2.2	Example 2	24
5.3	Case 3: Parsing Error & Semantic Analysis Aborted	25
5.3.1	Example 1	25
5.3.2	Example 2	26
6	Conclusion	26

1 Introduction

1.1 Problem Statement

The compilation process is divided into several phases. Following the lexical analysis, which decomposes the source code into a linear sequence of tokens, the compiler lacks structural information about how these tokens relate to each other. The primary challenge of the syntax analysis phase (parsing) is to transform this flat stream of tokens into a hierarchical structure that conforms to a specific Context-Free Grammar (CFG). Without an efficient parser, the compiler cannot validate the logical structure of programs or generate an Intermediate Representation (IR).

Beyond the technical challenge, there is a practical gap in education: the intermediate steps between raw source code and a validated parse tree are rarely visible to learners. Students often write code without understanding how a compiler processes it internally. ParseFlow, the product developed in this project, addresses both problems simultaneously — it implements a working compiler front-end and exposes each phase visually so the user can observe lexing, parsing, and semantic analysis in real time.

1.2 Motivation

The choice of an LALR(1) (Look-Ahead LR) parser is motivated by the need for a balance between parsing power and computational efficiency. While Canonical LR(1) parsers are extremely powerful, they often generate a prohibitive number of states for complex programming languages. LALR(1) provides a sophisticated optimization by merging states with identical cores, significantly reducing memory usage while retaining the ability to handle most modern programming language constructs. Implementing this from scratch in Java with a JavaFX graphical interface allows for a deep understanding of bottom-up parsing automata while also delivering a functional educational tool.

1.3 Objectives

- Implement a robust Java-based LALR(1) parser architecture built entirely from first principles.
- Develop an automated system for reading and processing Context-Free Grammars (CFG) from a plain-text file.
- Efficiently calculate *First* sets using a matrix-based fixed-point iteration algorithm. Note: FOLLOW sets are not computed explicitly because the LALR(1) algorithm derives lookaheads directly through the LR(1) closure operation, making a separate FOLLOW computation unnecessary.

- Construct the Canonical LR(1) collection of items and subsequently optimize it into an LALR(1) collection through state merging.
- Integrate a semantic analysis component that validates declarations, type compatibility, and scope rules via Syntax-Directed Translation (SDT).
- Deliver a complete JavaFX desktop application (ParseFlow) with a syntax-highlighted code editor, tabbed output panels, a zoomable parse tree visualization, and a light/-dark theme toggle.
- Ensure high modularity and maintainability of all components across the `lexer`, `parser`, `semantic`, and `ui` packages.

2 Theoretical Framework

2.1 Compiler Structure

If we examine the compilation process in more detail, we can see that it operates as a sequence of phases, each of which transforms one representation of the source program to another. [1] A typical decomposition of a compiler into phases is: Lexical analysis, Syntax analysis, Semantic analysis, Intermediate Code Generation, Code Optimization, and Target Code Generation. The first three phases are classified as the analysis phase and the last three are the synthesis phase. [1]

The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program. If the analysis part detects that the source program is either syntactically ill-formed or semantically unsound, it must provide informative messages so the user can take corrective action. The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part. [1]

The lexical analyzer classifies the source code into tokens and provides a stream of tokens to the syntax analyzer to validate the structure through a context-free grammar. The second phase of the compiler is syntax analysis, or parsing. The parser uses the first components of the tokens produced by the lexical analyzer to create a parse tree, which is a representation that illustrates the complete syntactic derivation according to the grammar used in the compiler. [5]

2.2 Context-Free Grammar

Context-Free Grammars (CFGs) play a central role in the syntax phase. The grammar is the structure that the language follows. In the context of compilers, it is important to understand the Chomsky hierarchy, one of the classifications used in computing theory.

Briefly, it has four classifications: type 0 places no restrictions on the production rules; type 1 has production rules that depend on the context of the symbols; type 2 has production rules that are independent of context; and type 3 uses regular expressions as a means of expressing patterns. [3]

Syntax analysis requires the use of type 2 grammars, also known as Context-Free Grammars. The basic structure of this grammar is:

$$A \rightarrow \alpha \quad (1)$$

Where:

$$A \in N$$

$$\alpha \in (N \cup T)^*$$

T : Terminal symbols

N : Non-terminal symbols

$*$: Star closure (zero or more symbols)

[6]

The left side consists of a non-terminal symbol that can be replaced by the right side, which is composed of non-terminal symbols, terminal symbols, or a combination of both. [2]

Grammars offer significant benefits for language designers and compiler writers:

- A grammar gives a precise, yet easy-to-understand, syntactic specification of a programming language. [1]
- From certain classes of grammars, we can construct automatically an efficient parser that determines the syntactic structure of a source program. [1]
- The parser-construction process can reveal syntactic ambiguities and trouble spots that might have slipped through the initial design phase of a language. [1]
- The structure imparted to a language by a properly designed grammar is useful for translating source programs into correct object code and for detecting errors. [1]
- A grammar allows a language to be evolved or developed iteratively by adding new constructs to perform new tasks. [1]

2.3 Context-Free Grammars in Compilers

To implement a Context-Free Grammar in a compiler, the produced grammar must be unambiguous. Ambiguity in a CFG occurs when, for a given input string, the grammar can produce more than one possible parse tree. To obtain an unambiguous CFG, each input string must correspond to exactly one parse tree. Some important elements in the implementation of a CFG within a compiler are the associativity and precedence of operators. [1]

2.4 Associativity

Associativity is related to recursion in a CFG. When a CFG contains left recursion — meaning a production rule generates the same non-terminal on the left side of the expression — the grammar is said to have left associativity. Similarly, when a CFG has right recursion, it is considered right-associative.

It is important to note that bottom-up (LR) parsers handle left recursion naturally and efficiently. In fact, the grammar implemented in this project makes deliberate use of left recursion in rules such as $\text{StmtList} \rightarrow \text{StmtList Stmt}$ and $\text{Expr} \rightarrow \text{Expr AddOp Term}$, because the LALR(1) engine reduces these iteratively without growing the stack exponentially. [1]

- $E \rightarrow E + T \mid T$ (Left recursion — preferred in bottom-up parsers)
- $E \rightarrow T + E \mid T$ (Right recursion)

2.5 Precedence

Precedence defines the order in which operators are evaluated. In a CFG, precedence can be represented by the depth of the parse tree. An operator that appears deeper in the tree has higher precedence, while an operator closer to the root has lower precedence. [1] For instance, in arithmetic expressions, multiplication typically has higher precedence than addition:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid id$

2.5.1 BNF

BNF (Backus–Naur Form) is a formal notation used to describe the syntax of programming languages and Context-Free Grammars in general. [4] Its objective is to represent, in a

structured and readable way, the production rules that define how the valid expressions of a language are constructed. The basic notation of a rule in BNF is: $\langle \text{symbol} \rangle ::= \langle \text{expression} \rangle$, where:

- **Non-terminals**: written between angle brackets ($\langle \rangle$), representing abstract syntactic categories.
- **Terminals**: the actual symbols of the language, such as keywords, operators, or punctuation marks.
- **Operator** $::=$: read as “is defined as” or “can be replaced by.”
- **Alternatives** ($|$): allow expressing options within the same production rule.

2.5.2 Types of Parsers

There are two main families of parsers:

- **Top-down parsers**: can use backtracking (Brute Force) or operate without backtracking (Recursive Descent, Predictive parsers, LL(1)).
- **Bottom-up parsers**: include operator precedence parsers and LR parsers (LR(0), SLR(1), LALR(1), CLR(1)).

[2]

Parsers typically operate with a stack, an input buffer, and a parsing table. [1] Top-down parsing constructs the parse tree starting from the root and creates nodes in preorder, which is equivalent to finding a leftmost derivation for the input string. [1]

Bottom-up parsing constructs the parse tree starting at the leaves and working upward toward the root. We can think of bottom-up parsing as the process of “reducing” a string w to the start symbol of the grammar. [1]

Parser construction is aided by two functions, **FIRST** and **FOLLOW**, associated with a grammar G . **FIRST**(α) is the set of terminals that can begin strings derived from α . **FOLLOW**(A) is the set of terminals that can appear immediately to the right of A in a sentential form. [1]

Although both FIRST and FOLLOW are relevant tools in certain parser families, the current implementation only computes FIRST sets. FOLLOW sets are not needed because the parser is based on LR(1)/LALR(1) items, where lookahead symbols are propagated directly through the closure operation. Unlike SLR(1) parsing — which uses FOLLOW sets as an approximation for lookaheads — our LALR(1) parser uses precise per-item lookaheads derived from the LR(1) construction and preserved during the merging step.

2.6 Semantic Analysis

We associate information with a language construct by attaching attributes to grammar symbols. A Syntax-Directed Definition (SDD) specifies the values of attributes by associating semantic rules with grammar productions. If X is a symbol and a is one of its attributes, then $X.a$ denotes the value of a at a particular parse-tree node labeled X . Attributes may be of any kind: numbers, types, table references, or strings. [1]

3 Product Overview: ParseFlow

ParseFlow is a JavaFX desktop application developed by RootNode Technologies that implements a complete compiler front-end for a C-like language. Its primary goal is educational: it makes every phase of compilation — lexical analysis, parsing, and semantic validation — visible and interactive in a single graphical interface.

3.1 Features

The following table summarizes the main capabilities of the final product:

Feature	Description
Syntax-highlighted editor	Real-time coloring by token type (keyword, id, constant, etc.)
LALR(1) parser	Built from scratch; no parser-generator libraries used
Lexer output tab	Shows each token with its type and lexeme
Parser output tab	Shows parsing steps and shift/reduce actions
Semantic output tab	Shows “SDT Verified!” or specific semantic error messages
Parse tree tab	Zoomable graphical parse tree rendered with JavaFX Canvas
Light / Dark theme	Toggle button switches between CSS themes at runtime
Fat JAR distribution	Packaged as <code>ParseFlow.jar</code> via Maven Shade Plugin

3.2 Token Normalization

The lexer produces tokens of types `KEYWORD`, `IDENTIFIER`, `OPERATOR`, `PUNCTUATOR`, `CONSTANT`, and `LITERAL`. Before tokens are passed to the parser, the `CompilerController` normalizes them as follows: keywords, operators, and punctuators are passed through with their original lexeme; identifiers are mapped to the grammar terminal `id`; numeric constants are mapped to `constant`; and string or character literals are mapped to `literal`. This normalization bridges the lexer’s detailed classification and the grammar’s abstract terminal symbols.

3.3 File Handling

When the user clicks Run, the application writes the editor content to `doc/test/test.c` and writes the normalized token stream to `doc/tokens/tokens.txt`. The grammar is loaded from `doc/grammar/grammar.txt`. This allows the files to be inspected externally after each run.

4 Development

The development of the parser was structured as a modular Java application, designed to bridge the gap between abstract Context-Free Grammar (CFG) theory and functional syntactic analysis. The system is organized into four packages within `mx.unam.fi.compilers.g5.team03`: `lexer`, `parser`, `semantic`, and `ui`. The project follows an object-oriented design to manage the mathematical complexity of the LALR(1) algorithm.

4.1 Architectural Overview

The system architecture is divided into four functional layers:

1. **Data Modeling Layer:** Classes that represent the atomic units of a grammar (`Production`, `Item`, `State`, `Transition`).
2. **Processing Layer:** Logic for pre-calculating sets and transitions (`FirstSet`, `Closure`, `Goto`).
3. **Orchestration Layer:** Classes that build the DFA and optimize it (`CanonicalCollection`, `LALRCollection`, `Grammar`).
4. **User Interface Layer:** The JavaFX application (`CompilerApp`, `MainView`, `CompilerController`, `ParseTreePane`).

4.2 Core Component Modeling

4.2.1 Atomic Units: Productions and Items

The `Production.java` class serves as the fundamental data structure. Each instance stores a unique identifier, a Left-Hand Side (LHS) representing a non-terminal, and a list of strings for the Right-Hand Side (RHS).

Building upon this, `Item.java` implements the concept of an **LR(1) Item**. An item is a production with a “dot” (·) indicating how much of the production has been seen so far, along with a *lookahead* terminal.

4.2.2 The DFA State Machine

The automaton’s nodes are represented by `State.java`. A state is a unique collection of items. This class includes logic to compare states based on their “core” (the productions and dot positions, ignoring lookaheads), which is the cornerstone of LALR optimization. The `Transition.java` class acts as an edge in the DFA, mapping a transition symbol to a target state ID.

4.3 Grammar Pre-processing

4.3.1 Grammar Loading and Identification

The `Grammar.java` class handles I/O operations. It parses a text file (`doc/grammar/grammar.txt`) where each line follows a BNF-like format. The class automatically categorizes symbols: any symbol appearing on the LHS of a production is marked as a **Non-Terminal**, while all remaining symbols are classified as **Terminals**.

4.3.2 FIRST Set Calculation

The `FirstSet.java` class implements a fixed-point iteration algorithm to determine the set of terminals that can begin a string derived from any symbol. It uses a boolean matrix where rows represent symbols and columns represent terminals, updating iteratively until no further changes occur.

4.4 DFA Construction Logic

4.4.1 The Closure Operation

The `Closure.java` class expands a set of items. When an item has the dot before a non-terminal B , all productions of B are added to the state. The new items for B receive a lookahead calculated from $\text{FIRST}(\beta a)$, where β is the string following B in the original item and a is the original item's lookahead. This ensures correct LR(1) lookahead propagation.

4.4.2 The Goto Operation

Implemented in `Goto.java`, this function calculates the state reached by “consuming” a symbol: it moves the dot forward in all matching items and then computes the `Closure` of the resulting set.

4.5 Collection Orchestration and Optimization

4.5.1 Canonical LR(1) Collection

The `CanonicalCollection.java` class constructs the complete set of reachable LR(1) states starting from the augmented grammar production. The implementation iterates over a dynamically growing list of discovered states. For each state, it determines all grammar symbols that appear immediately after a dot, applies the Goto operation for each such symbol, and adds any newly generated states to the list. This continues until no new states are discovered, producing the full canonical LR(1) collection.

4.5.2 LALR(1) Merging (The Core Logic)

The `LALRCollection.java` class performs the crucial optimization step. It identifies groups of LR(1) states that share the same core — identical sets of (production, dot position) pairs, disregarding lookaheads — and merges them into single LALR(1) states. Lookaheads from all merged states are unioned together. The resulting state mapping is tracked via an `oldToNew[]` array used to remap all transitions.

4.6 Parsing Table Synthesis

The `ParseTable.java` class constructs the numeric matrix that enables constant-time action lookups during parsing.

- **ACTION Table:** Maps a state and a lookahead terminal to one of four outcomes: Shift (positive integer — target state), Reduce (negative integer — production number to apply), Accept (999), or Error (0).
- **GOTO Table:** Maps a state and a non-terminal to the next target state after a reduction.

Any attempt to write conflicting entries raises an `IllegalStateException`, ensuring the grammar is verified as conflict-free during table construction.

4.7 The Shift-Reduce Parsing Engine

The `ShiftReduceParser.java` class reads the normalized token stream and processes it using a pushdown automaton architecture. The parser maintains three parallel stacks: the **State Stack**, the **Symbol Stack**, and the **Tree Stack**.

The State Stack stores the numeric states used to consult the ACTION and GOTO tables. The Symbol Stack stores the grammar symbols that have been shifted or reduced during the analysis. The Tree Stack stores `ParseTreeNode` objects, which are assembled into a parse tree as productions are applied during the reduction process.

It is important to clarify that this structure produces a **parse tree**, not an Abstract Syntax Tree (AST). The parse tree preserves the complete syntactic derivation from the grammar, including all intermediate non-terminals. An AST would simplify or eliminate grammatical detail to retain only the semantically relevant structure. In this implementation, the parse tree is used both for semantic analysis and for graphical visualization.

4.7.1 Parse Tree Visualization

After parsing is complete, the system renders the parse tree in the **Parse Tree** tab using `ParseTreePane.java`, a custom JavaFX component. The tree is drawn on a scrollable

canvas with zoom support (via the – and + buttons and a Reset option). Each node displays the grammar symbol it represents, and terminal nodes additionally show their lexeme (e.g., constant : 1).

For the following input:

```
int main() {
    return 1;
}
```

The resulting parse tree preserves all intermediate grammar symbols such as Expr, Term, Unary, and Primary, confirming that the structure is a complete parse tree and not an AST.

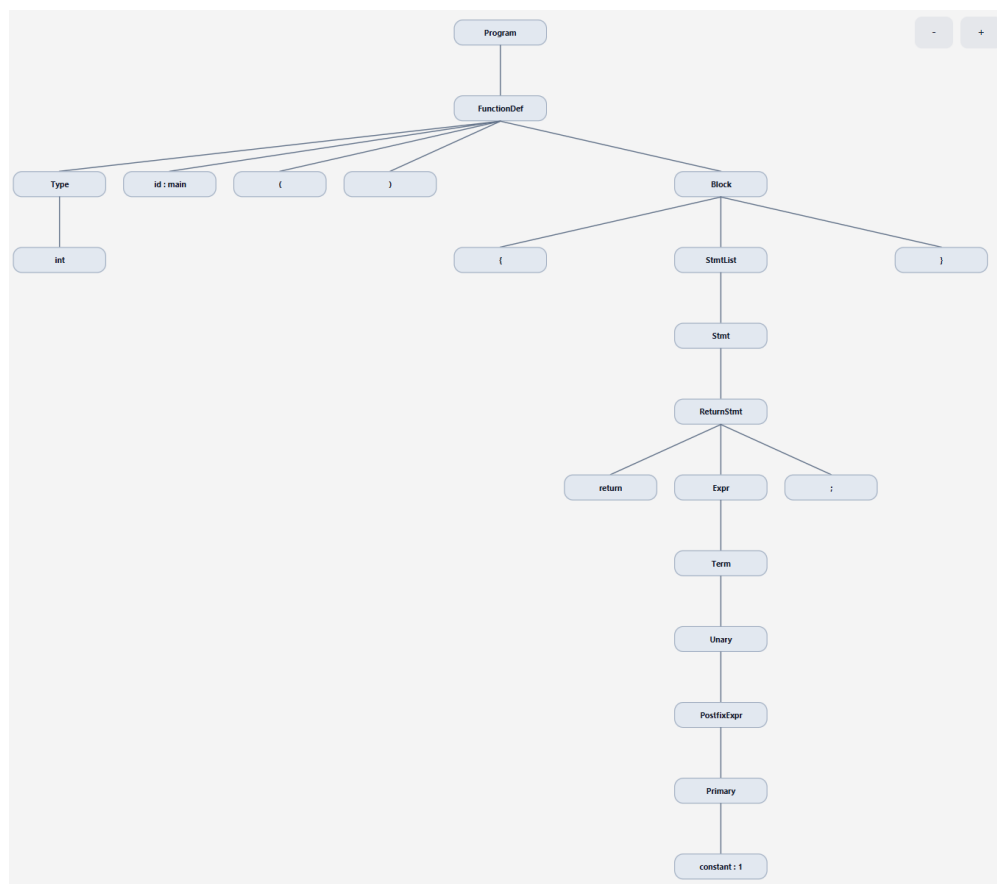


Figure 1: Parse tree for `int main() { return 1; }`

4.8 Grammar Implementation Details

The parser relies on a carefully designed 65-rule Context-Free Grammar that defines the structure of the supported C-like language. The explicit formulation of this grammar is critical, as its structure directly dictates the behavior of the parsing engine.

0	Program'	-> Program
1	Program	-> Program FunctionDef
2	Program	-> FunctionDef
3	FunctionDef	-> Type id () Block
4	Type	-> int
5	Type	-> char
6	Type	-> void
7	Block	-> { }
8	Block	-> { StmtList }
9	StmtList	-> StmtList Stmt
10	StmtList	-> Stmt
11	Stmt	-> DeclStmt
12	Stmt	-> AssignStmt
13	Stmt	-> IfStmt
14	Stmt	-> PostfixStmt
15	Stmt	-> CallStmt
16	Stmt	-> ReturnStmt
17	Stmt	-> Block
18	DeclStmt	-> Type InitDeclList ;
19	InitDeclList	-> InitDeclList , InitDecl
20	InitDeclList	-> InitDecl
21	InitDecl	-> id
22	InitDecl	-> id = Expr
23	AssignStmt	-> LValue = Expr ;
24	LValue	-> id
25	IfStmt	-> if (Condition) Block
26	IfStmt	-> if (Condition) Block else Block
27	PostfixStmt	-> id ++ ;
28	PostfixStmt	-> id -- ;
29	CallStmt	-> id () ;
30	CallStmt	-> id (ArgList) ;
31	ArgList	-> ArgList , Expr
32	ArgList	-> Expr
33	ReturnStmt	-> return Expr ;
34	Condition	-> Expr RelOp Expr
35	Condition	-> Expr
36	RelOp	-> >
37	RelOp	-> <
38	RelOp	-> >=
39	RelOp	-> <=
40	RelOp	-> ==
41	RelOp	-> !=
42	Expr	-> Expr AddOp Term
43	Expr	-> Term
44	AddOp	-> +
45	AddOp	-> -
46	Term	-> Term MulOp Unary

```

47 Term          -> Unary
48 MulOp         -> *
49 MulOp         -> /
50 MulOp         -> %
51 Unary         -> UnaryOp Unary
52 Unary         -> PostfixExpr
53 UnaryOp       -> +
54 UnaryOp       -> -
55 UnaryOp       -> !
56 UnaryOp       -> &
57 UnaryOp       -> *
58 PostfixExpr   -> id ++
59 PostfixExpr   -> id --
60 PostfixExpr   -> Primary
61 Primary       -> id
62 Primary       -> constant
63 Primary       -> literal
64 Primary       -> ( Expr )

```

Listing 1: Complete 65-Rule Grammar Specification

4.8.1 Grammar Structural Analysis

The design of Listing 1 solves several classic parsing challenges:

- **Left-Recursion for Lists:** Rules like `StmtList -> StmtList Stmt` (Rule 9) and `Expr -> Expr AddOp Term` (Rule 42) use left-recursion deliberately. The LALR(1) bottom-up parser handles left-recursion efficiently: it reduces each item as it is encountered, keeping the stack depth bounded regardless of how many statements or operands appear in sequence.
- **Strict Operator Precedence:** To prevent ambiguity in mathematical expressions, the grammar enforces a strict derivation hierarchy: `Expr` (addition/subtraction) delegates to `Term` (multiplication/division/modulo), which delegates to `Unary`, which delegates to `PostfixExpr` and finally to `Primary`. This guarantees that `x + y * 3` is parsed with multiplication binding more tightly than addition.
- **The Dangling-Else Ambiguity:** Rules 25 and 26 introduce the classic dangling-else problem. The LALR(1) lookahead mechanism resolves this conflict in favor of **Shift** — associating the `else` with the nearest `if` — which matches standard C semantics.

4.9 Semantic Rules Specification

The file `doc/semantic/semantic_rules.txt` defines the Attribute Grammar used for semantic validation. Key rules implemented include:

- **Variable Declaration:** Ensures identifiers are inserted into the symbol table upon declaration and that no identifier is declared twice in the same scope (`lookupCurrent(id.lexeme)`).
- **Type Checking:** For assignments and expressions, the analyzer verifies type compatibility (e.g., `compatible(t1, t2)` for arithmetic operations, with `numeric(t)` checking membership in `{int, char}`).
- **Function Scope:** `insertFunc` manages function signatures, ensuring the return type matches the `return` statement's expression type and that a main function is defined.

4.9.1 Syntax-Directed Translation Validation

The semantic validation follows the principles of Syntax-Directed Translation (SDT), where semantic actions are associated with grammar productions. During the reduction process, the `SemanticAnalyzer` traverses the parse tree and executes the corresponding validation rule for each node. This connects syntactic structure to semantic checks such as declaration validation, type compatibility, function return verification, and scope control. As a result, a program that passes both stages is not only syntactically valid but also semantically coherent according to the rules of the language.

4.9.2 Semantic Analysis Component

The semantic analysis component is implemented in the `semantic` package and includes `SemanticAnalyzer.java`, `SymbolTable.java`, `VariableSymbol.java`, `FunctionSymbol.java`, and `SemanticError.java`. The `SymbolTable` supports scoped lookup (`lookup` searches all active scopes; `lookupCurrent` searches only the innermost scope). The analyzer collects all detected errors into a list and returns them to the `CompilerController`, which displays them in the Semantic Output tab.

4.10 User Interface: ParseFlow Application

The graphical interface is built with JavaFX and the `RichTextFX` library for syntax highlighting. The entry point is `Main.java`, which launches `CompilerApp`. Layout and components are constructed programmatically in `MainView.java`, and all application logic is handled by `CompilerController.java`.

The interface consists of a **top bar** with a Run button, a Light Mode/Dark Mode toggle, and the RootNode Technologies and ParseFlow logos on the right. Below the top bar, a SplitPane divides the window into a left panel containing the code editor and a right panel containing a TabPane with four tabs: Lexer Output, Parser Output, Semantic Output, and Parse Tree.

The code editor uses RichTextFX's CodeArea, which supports real-time syntax highlighting. As the user types, tokens are classified by the lexer and styled with CSS classes (token-keyword, token-identifier, token-constant, etc.). Two CSS stylesheets — dark-theme.css and light-theme.css — control the complete visual appearance, switching at runtime when the user toggles the theme button.

4.11 Main Execution Flow

The CompilerController.runCompiler() method orchestrates the complete front-end process when the user clicks Run:

```
// 1. Read source code from the editor
String sourceCode = view.getCodeEditor().getText();

// 2. Write source code to doc/test/test.c
writeTest(sourceCode);

// 3. Execute the lexical analyzer
Lexer lexer = new Lexer();
List<Token> tokens = lexer.tokenize(sourceCode);

// 4. Normalize tokens for the parser and write doc/tokens/tokens.txt
List<ParserToken> parserTokens = normalizeTokensForParser(tokens);
writeTokens(tokensFileContent);
view.getLexerOutput().setText(lexerText);

// 5. Load Grammar from doc/grammar/grammar.txt and compute FIRST sets
Grammar grammar = new Grammar(GRAMMAR_FILE_PATH.toString());
FirstSet fs = new FirstSet(grammar);

// 6. Build the full LR(1) DFA
Closure closure = new Closure(grammar, fs);
Goto goTo = new Goto(closure);
CanonicalCollection lr = new CanonicalCollection(grammar, closure,
    goTo);

// 7. Optimize the DFA into LALR(1)
LALRCollection lalr = new LALRCollection(lr.getStates());

// 8. Synthesize the Parse Table
```

```

ParseTable parseTable = new ParseTable(grammar, lalr.getStates());

// 9. Run the shift-reduce parser and build the parse tree
ShiftReduceParser parser = new ShiftReduceParser(grammar, parseTable)
;
ParseTreeNode root = parser.parse(parserTokens);
view.getParserOutput().setText(buildParserOutput(parser));

// 10. Execute semantic validation
SemanticAnalyzer analyzer = new SemanticAnalyzer();
boolean ok = analyzer.analyze(root);
view.getSemanticOutput().setText(ok ? "SDT Verified!" : formatErrors(
    analyzer));

// 11. Render the parse tree in the Parse Tree tab
view.getParseTreePane().setTree(root);

```

Listing 2: Execution flow in `CompilerController.runCompiler()`

5 Results

The ParseFlow application is accessed through its graphical interface. The user enters source code in the editor panel — with real-time syntax highlighting — and clicks the **Run** button to process it. Results are displayed across four tabs: Lexer Output, Parser Output, Semantic Output, and Parse Tree. The Parser Output shows a summary of the parsing process along with the most significant shift and reduce actions applied. The Parse Tree tab renders the full syntactic derivation as an interactive, zoomable tree.

5.1 Case 1: Parsing Success & SDT Verified

5.1.1 Example 1

The screenshot shows a web-based parser interface. At the top, there are two buttons: "Run" (blue) and "Light Mode" (grey). Below the buttons is a code editor containing the following C code:

```
int main() {  
    return 0;  
}
```

To the right of the code editor is a panel with four tabs: "Lexer Output", "Parser Output" (selected), "Semantic Output", and "Parse Tree". The "Parser Output" tab displays the following text:

Parsing Success!

Total steps: 22
Parse completed without syntax errors.

The output continues with a list of parser actions:

- Shift int
- Reduce by Type -> int
- Shift id
- Shift (
- Shift)
- Shift {
- Shift return
- Shift constant
- Reduce by Primary -> constant
- Reduce by PostfixExpr -> Primary
- Reduce by Unary -> PostfixExpr
- Reduce by Term -> Unary
- Reduce by Expr -> Term
- Shift ;
- Reduce by ReturnStmt -> return Expr ;
- Reduce by Stmt -> ReturnStmt
- Reduce by StmtList -> Stmt
- Shift }
- Reduce by Block -> { StmtList }
- Reduce by FunctionDef -> Type id () Block
- Reduce by Program -> FunctionDef
- Accept

Figure 2: Parsing Success

This screenshot shows the same parser interface as Figure 2, but with the "Parser Output" tab displaying "SDT Verified!". The code in the editor is the same: `int main() { return 0; }`. The "Run" and "Light Mode" buttons are still present at the top.

Figure 3: SDT Verified

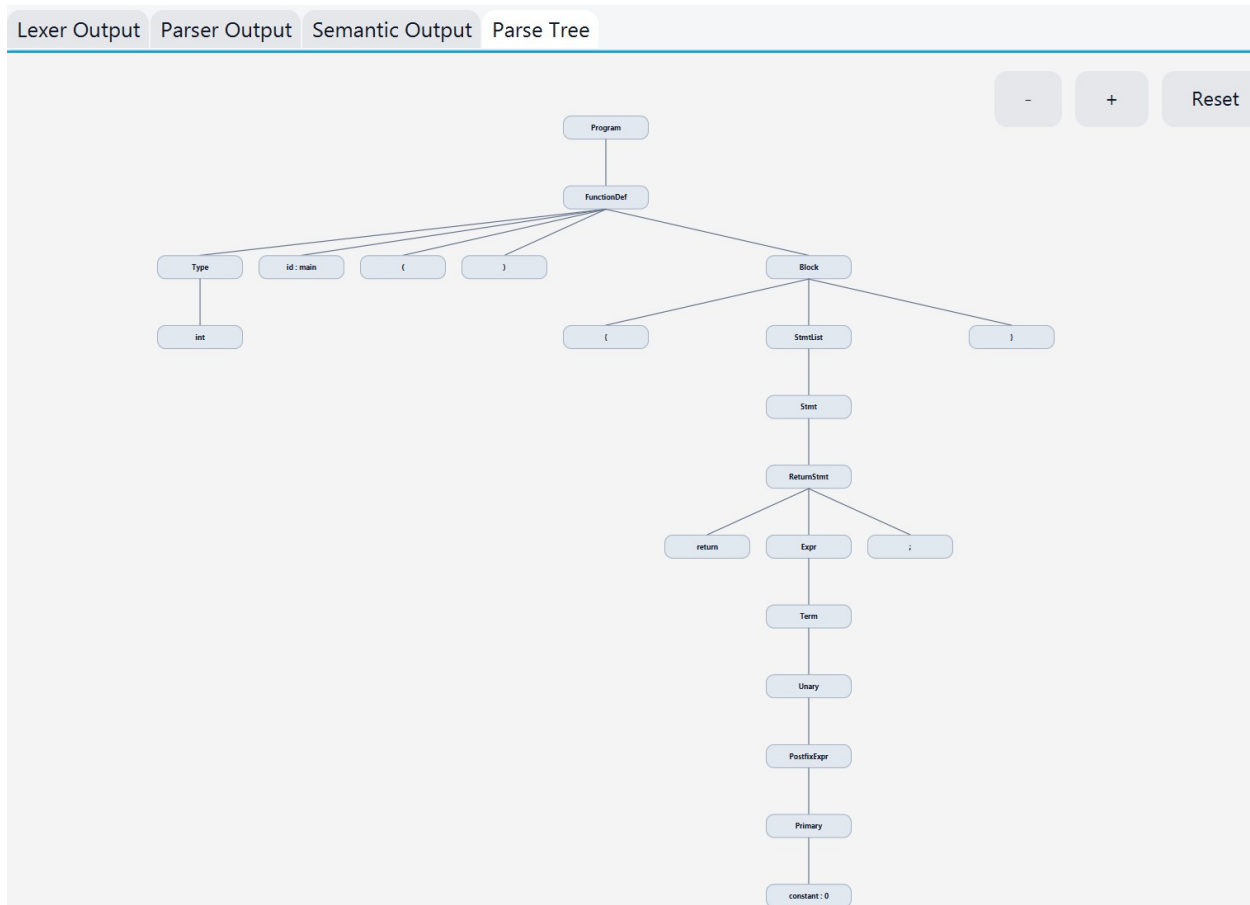


Figure 4: Parse Tree

Explanation: The lexical analyzer produces tokens such as `int`, `id:main`, `(`, `)`, `{`, `return`, `constant:0`, `;`, `}`, and `$`. These are normalized and passed to the parser. The LALR(1) parser uses the ACTION/GOTO table — built from the canonical LR(1) collection and merged LALR states — to perform a bottom-up derivation without backtracking, applying shift and reduce actions until it reaches the accept state. The parse tree is then constructed from the sequence of reductions.

Next, the semantic analyzer traverses the parse tree using SDT rules. It verifies that `main` is defined in the function table and that the return type (`int`) is compatible with the returned value (`0`, of type `int`). No errors are found, and the Semantic Output tab displays “SDT Verified!”.

5.1.2 Example 2

The screenshot shows a web-based parser interface. On the left, a code editor contains the following C-like code:

```
int main() {  
    int a;  
    a = 10;  
    if (a > 5) {  
        a = 1;  
    } else {  
        a = 2;  
    }  
    return a;  
}
```

On the right, there are four tabs: "Lexer Output", "Parser Output" (which is selected), "Semantic Output", and "Parse Tree". The "Parser Output" tab displays the following text:

Parsing Success!

Total steps: 98
Parse completed without syntax errors.

The output also lists a series of shift and reduce actions performed by the parser, such as "Shift int", "Reduce by Type -> int", "Shift id", "Shift (", "Shift)", "Shift {", "Shift int", "Reduce by Type -> int", "Shift id", "Reduce by InitDecl -> id", "Reduce by InitDeclList -> InitDecl", "Shift ;", "Reduce by DeclStmnt -> Type InitDeclList ;", "Reduce by Stmnt -> DeclStmnt", "Reduce by StmntList -> Stmnt", "Shift id", "Reduce by lValue -> id", "Shift =", and "Shift constant".

Figure 5: Parsing Success

This screenshot shows the same parser interface as Figure 5, but with the "Semantic Output" tab selected. The output displays "SDT Verified!", indicating that the code has passed semantic checks.

Figure 6: SDT Verified

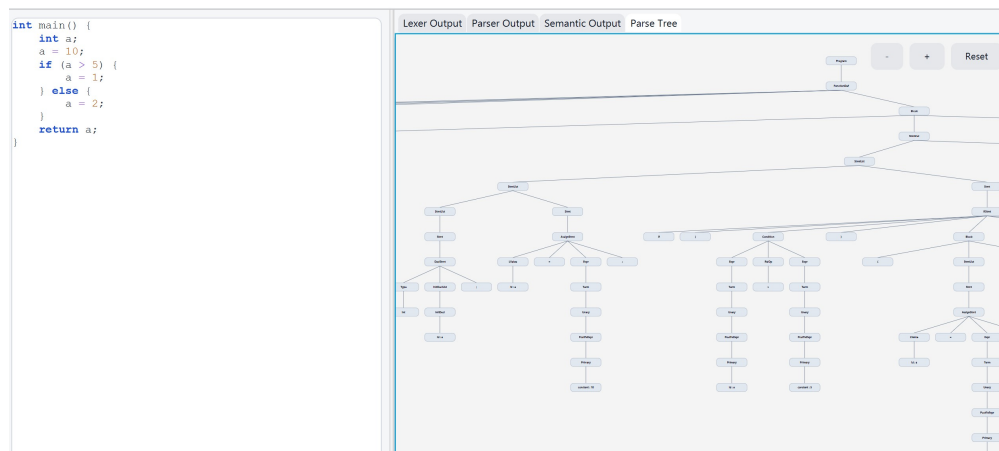


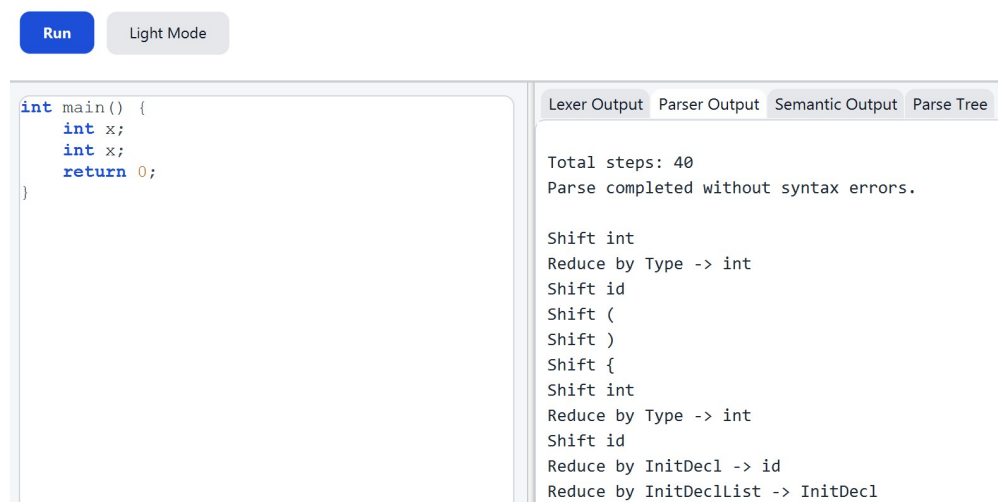
Figure 7: Parse Tree

Explanation: The lexer transforms the source code into a sequence of tokens (identifiers, constants, operators, and reserved words). The LALR(1) parser processes this sequence using grammar constructs such as DeclStmt, AssignStmt, IfStmt, and ReturnStmt, following the ACTION/GOTO table without conflicts and producing a valid parse tree.

During the semantic phase, the SDT inserts declared variables into the symbol table, checks for redeclarations, and validates type consistency in assignments. The if condition is verified to ensure it contains a valid relational expression. Since no errors are found, the analysis concludes with “SDT Verified!”.

5.2 Case 2: Parsing Success & SDT Error

5.2.1 Example 1



The screenshot shows a compiler interface with a source code editor on the left and a multi-tabbed output window on the right. The source code is a C program: `int main() { int x; int x; return 0; }`. The output window has tabs for 'Lexer Output', 'Parser Output', 'Semantic Output', and 'Parse Tree'. The 'Parser Output' tab is active, displaying the following text: 'Total steps: 40', 'Parse completed without syntax errors.', and a list of parser actions: 'Shift int', 'Reduce by Type -> int', 'Shift id', 'Shift (', 'Shift)', 'Shift {', 'Shift int', 'Reduce by Type -> int', 'Shift id', 'Reduce by InitDecl -> id', and 'Reduce by InitDeclList -> InitDecl'.

```
int main() {
    int x;
    int x;
    return 0;
}
```

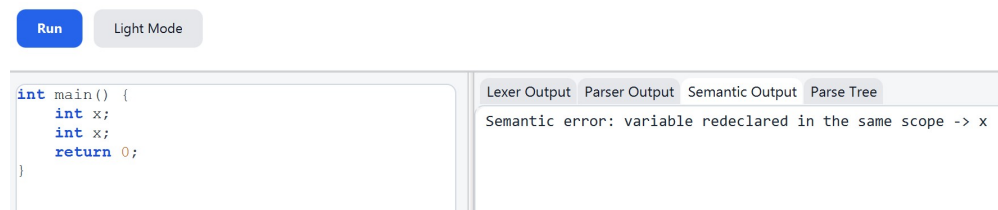
Run Light Mode

Lexer Output Parser Output Semantic Output Parse Tree

Total steps: 40
Parse completed without syntax errors.

Shift int
Reduce by Type -> int
Shift id
Shift (
Shift)
Shift {
Shift int
Reduce by Type -> int
Shift id
Reduce by InitDecl -> id
Reduce by InitDeclList -> InitDecl

Figure 8: Parsing Success



The screenshot shows the same compiler interface as Figure 8, but with a semantic error. The source code is identical: `int main() { int x; int x; return 0; }`. The 'Semantic Output' tab is active, displaying the message: 'Semantic error: variable redeclared in the same scope -> x'.

```
int main() {
    int x;
    int x;
    return 0;
}
```

Run Light Mode

Lexer Output Parser Output Semantic Output Parse Tree

Semantic error: variable redeclared in the same scope -> x

Figure 9: SDT Error

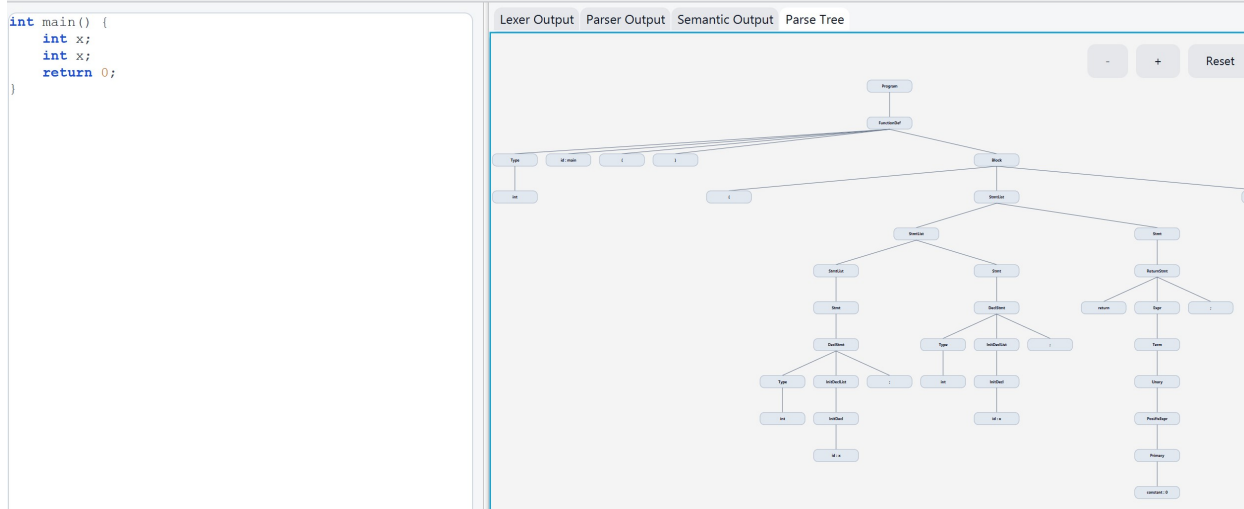


Figure 10: Parse Tree

Explanation: Lexical and syntactic analysis complete successfully because the grammar permits any number of declaration statements — it does not enforce name uniqueness. The parser reduces both `int x;` declarations as separate `DeclStmt` nodes, producing a valid parse tree.

During the semantic phase, the SDT attempts to insert each occurrence of `x` into the current scope. On the second insertion, `lookupCurrent` finds that the identifier already exists and adds a semantic error: “variable redeclared in the same scope”. Although parsing succeeds, the Semantic Output displays the error, demonstrating that correct syntax alone does not guarantee a valid program.

5.2.2 Example 2

RunLight Mode

```
int main() {  
    int y;  
    y = "hola";  
    return y;  
}
```

Lexer OutputParser OutputSemantic OutputParse Tree

Parsing Success!

Total steps: 44
Parse completed without syntax errors.

Shift int
Reduce by Type -> int
Shift id
Shift (
Shift)
Shift {
Shift int
Reduce by Type -> int
Shift id
Reduce by InitDecl -> id
Reduce by InitDeclList -> InitDecl

Figure 11: Parsing Success

RunLight Mode

```
int main() {  
    int y;  
    y = "hola";  
    return y;  
}
```

Lexer OutputParser OutputSemantic OutputParse Tree

Semantic error: incompatible assignment. Left side is int but expression is string.

Figure 12: SDT Error

```
int main() {  
    int y;  
    y = "hola";  
    return y;  
}
```

Lexer OutputParser OutputSemantic OutputParse Tree

Figure 13: Parse Tree

Explanation: The lexer classifies "hola" as a LITERAL token, which is normalized to the grammar terminal `literal`. The grammar allows literals within expressions (Rule 63: `Primary -> literal`), so `y = "hola";` is syntactically valid and the LALR(1) parser builds the parse tree without errors.

During semantic analysis, the SDT infers the right-hand side as type `string` (a string literal enclosed in double quotes), while `y` was declared as `int`. Since `compatible(int, string)` is false, a semantic error is generated: "incompatible assignment. Left side is int but expression is string". This confirms that the SDT correctly detects type violations even when syntax is valid.

5.3 Case 3: Parsing Error & Semantic Analysis Aborted

5.3.1 Example 1

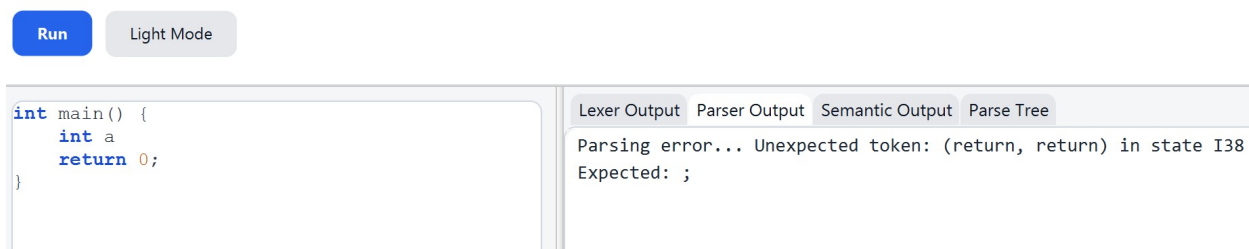


Figure 14: Parsing Error

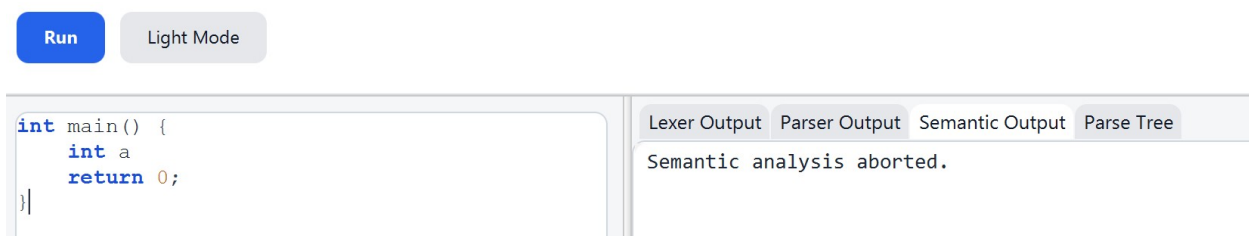


Figure 15: Semantic Analysis Aborted

Explanation: The lexer generates the token sequence and the parser begins processing using the ACTION table. When reducing `DeclStmt`, the grammar requires a terminating `;`, but the lookahead token is `return`. Since no valid ACTION entry exists for this combination in the current state, the parser throws an `IllegalStateException` reporting the unexpected token and the state in which the error occurred.

Because parsing does not complete, no parse tree is produced and the semantic analyzer is never invoked. The Semantic Output displays "Semantic analysis aborted.". This case demonstrates correct detection of invalid token sequences by the parsing table.

5.3.2 Example 2

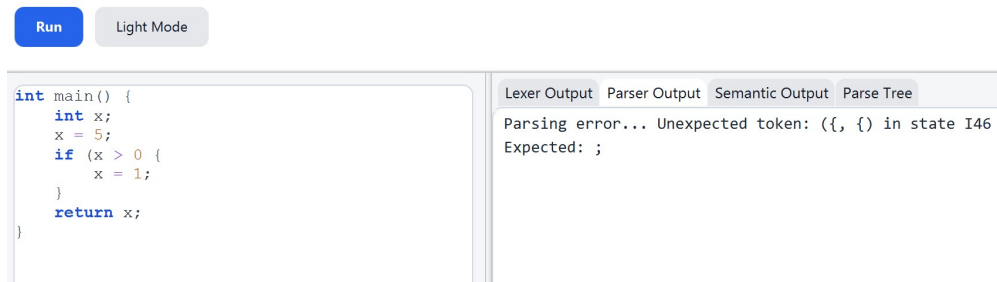


Figure 16: Parsing Error

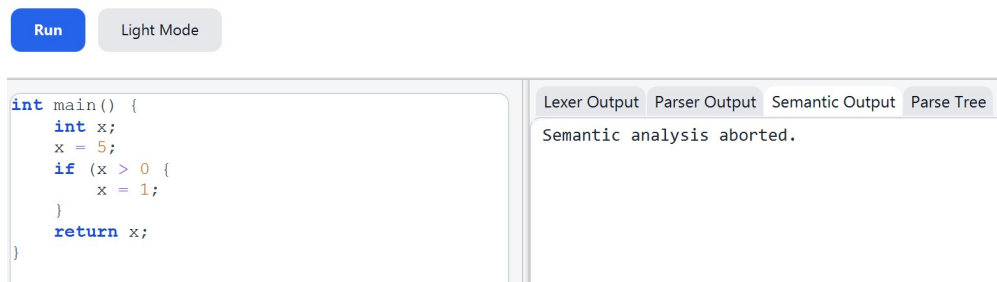


Figure 17: Semantic Analysis Aborted

Explanation: The token sequence is processed correctly up to `if ( x > 0`. The grammar requires a closing `)` to complete the condition (Rule 34: `Condition -> Expr RelOp Expr`), but the next token is `{`, leading to an invalid transition. With no valid ACTION entry, the parser reports a syntax error and stops. Semantic analysis is not executed, and the Semantic Output displays "Semantic analysis aborted."

6 Conclusion

The development of the LALR(1) parser together with the semantic analyzer and the ParseFlow graphical interface made it possible to apply several key concepts of compiler design in a practical setting. Throughout the project, it became evident how a context-free grammar defines the structure of a language and how it can be transformed into a working parsing mechanism that processes a stream of tokens into a structured representation.

The computation of FIRST sets, the closure operation, and the construction of the canonical LR(1) collection provided a solid theoretical basis. The transition to LALR(1) by merging states with identical cores highlighted an important trade-off: reducing the number of states while preserving enough lookahead information to make correct reduction decisions. This step illustrated how real compilers balance parsing power and memory efficiency.

From an implementation perspective, organizing the system into four distinct packages `lexer`, `parser`, `semantic`, and `ui` proved highly effective. Dividing responsibilities between modules such as grammar loading, parsing table construction, the shift-reduce engine, and the graphical interface made the system easier to understand and maintain independently. The use of three parallel stacks (states, symbols, and parse tree nodes) clarified how the shift-reduce automaton operates internally.

The inclusion of semantic validation through SDT extended the parser into a complete compiler front-end. The symbol table with scope management and type-checking rules made it possible to detect redeclarations, undeclared variables, and type incompatibilities, demonstrating that correct syntax alone does not guarantee a valid program.

The graphical parse tree visualization was one of the most pedagogically valuable components. By rendering the full derivation, including all intermediate non-terminals such as `Expr`, `Term`, `Unary`, and `Primary` the application makes abstract grammar rules concrete and observable. The light/dark theme, syntax-highlighted editor, and tabbed output panels contribute to a coherent user experience suitable for both classroom use and self-study.

Overall, this project demonstrated how formal compiler theory is applied in practice to transform a sequence of characters into a validated, structured representation, and how wrapping that pipeline in an interactive visual tool makes the process accessible and educational.

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson Education, 2007.
- [2] K. C. Loudon, *Compiler Construction: Principles and Practice*. Cengage Learning, 1997.
- [3] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Addison-Wesley, 2006.
- [4] J. Backus, "The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM Conference," 1959.
- [5] GeeksforGeeks, "Working of lexical analyzer in compiler," [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/working-of-lexical-analyzer-in-compiler/>.
- [6] GeeksforGeeks, "Context Free Grammar," [Online]. Available: <https://www.geeksforgeeks.org/context-free-grammar/>.
- [7] GeeksforGeeks, "LR Parser," Jul. 23, 2025. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/lr-parser/>. [Accessed: Apr. 30, 2026].

- [8] GeeksforGeeks, "LALR Parser (with Examples)," Jun. 24, 2021. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/lalr-parser-with-examples/>. [Accessed: Apr. 30, 2026].
- [9] GeeksforGeeks, "Parse Tree in Compiler Design," Jul. 15, 2025. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/parse-tree-in-compiler-design/>. [Accessed: Apr. 30, 2026].
- [10] GeeksforGeeks, "Semantic Analysis in Compiler Design," Apr. 11, 2026. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>. [Accessed: Apr. 30, 2026].
- [11] Tutorialspoint, "Compiler Design - Quick Guide." [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_quick_guide.htm. [Accessed: Apr. 30, 2026].
- [12] OpenJDK, "openjdk/jfx, branch jfx26," GitHub. [Online]. Available: <https://github.com/openjdk/jfx/tree/jfx26>. [Accessed: Apr. 30, 2026].
- [13] OpenJFX, "JavaFX CSS Reference Guide," JavaFX 26. [Online]. Available: <https://openjfx.io/javadoc/26/javafx.graphics/javafx/scene/doc-files/cssref.html>. [Accessed: Apr. 30, 2026].
- [14] OpenJFX, "JavaFX Software Development Kit, Version 26 API Specification," JavaFX 26. [Online]. Available: <https://openjfx.io/javadoc/26/>. [Accessed: Apr. 30, 2026].
- [15] OpenJFX, "Getting Started with JavaFX." [Online]. Available: <https://openjfx.io/openjfx-docs/#introduction>. [Accessed: Apr. 30, 2026].